

Digital Image Processing

Homework 3

PART1:

3.22 Answer the following:

- (a)* If $\mathbf{v} = [1 \ 2 \ 1]^T$ and $\mathbf{w}^T = [2 \ 1 \ 1 \ 3]$, is the kernel formed by $\mathbf{vw}^T$ separable?
- (b) The following kernel is separable. Find w_1 and w_2 such that $w = w_1 \star w_2$.

$$w = \begin{bmatrix} 1 & 3 & 1 \\ 2 & 6 & 2 \end{bmatrix}$$

(a) $\mathbf{vw}^T = \begin{bmatrix} 1 \\ 2 \\ 1 \end{bmatrix} [2 \ 1 \ 1 \ 3] = \begin{bmatrix} 2 & 1 & 1 & 3 \\ 4 & 2 & 2 & 6 \\ 2 & 1 & 1 & 3 \end{bmatrix}$ for $V_{3 \times 4}$ and $W_{1 \times 4}^T$, each row/col can be scalar multiple of any row/col, and $\mathbf{vw}^T$ is outer product of $\mathbf{v}$ and $\mathbf{w}$ #

(b) $\begin{bmatrix} 1 & 3 & 1 \\ 2 & 6 & 2 \end{bmatrix} = W_1 \star W_2 = \begin{bmatrix} 1 \\ 2 \end{bmatrix} [1 \ 3 \ 1]$, $W_1 = \begin{bmatrix} 1 \\ 2 \end{bmatrix}$, $W_2 = [1 \ 3 \ 1]$ #

3.28 An image is filtered with three Gaussian lowpass kernels of sizes 3×3 , 5×5 , and 7×7 , and standard deviations 1.5, 2, and 4, respectively. A composite filter, w , is formed as the convolution of these three filters.

- (a)* Is the resulting filter Gaussian? Explain.
- (b) What is its standard deviation?
- (c) What is its size? $(n_1-1) + (n_2-1) + (n_3-1) + 1$

(a) Yes, the convolution of gaussian remains gaussian. #

(b) $\sigma_w = \sqrt{1.5^2 + 2^2 + 4^2} = 4.72$ #

(c) $(3+5)-1 = 7$, $(7+7)-1 = 13$
 $\therefore$ composite size = 13×13 #

3.44 Are any of the following highpass (sharpening) kernels separable? For those that are, find vectors $\mathbf{v}$ and $\mathbf{w}$ such that $\mathbf{vw}^T$ equals the kernel(s).

- (a) The Laplacian kernels in Figs. 3.45(a) and (b).
 (b) The Roberts cross-gradient kernels shown in Figs. 3.50(b) and (c).
 (c)* The Sobel kernels in Figs. 3.50(d) and (e).

a)

$$\begin{bmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{bmatrix} \quad \begin{bmatrix} 1 & 1 & 1 \\ 1 & -8 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

not separable not separable

not proportional

b)

$$\begin{bmatrix} -1 & 0 \\ 0 & 1 \end{bmatrix} \quad \begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix}$$

not separable not separable

c)

$$\begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix} \quad \begin{bmatrix} -1 & 0 & 1 \\ 2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}$$

separable separable

$$\begin{bmatrix} -1 \\ 0 \\ 1 \end{bmatrix} \begin{bmatrix} 1 & 2 & 1 \end{bmatrix} \quad \begin{bmatrix} -1 \\ 2 \\ -1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 1 \end{bmatrix}$$

4.3 What is the convolution of two, 1-D impulses:

- (a)* $\delta(t)$ and $\delta(t - t_0)$?
 (b) $\delta(t - t_0)$ and $\delta(t + t_0)$?

a)

$$\delta(t) * \delta(t - t_0) = \int_{-\infty}^{\infty} \delta(\tau) \delta(t - t_0 - \tau) d\tau = \mathcal{F}^{-1}[1 \cdot e^{-j\omega t_0}] = \delta(t - t_0)$$

b)

$$\delta(t - t_0) * \delta(t + t_0) = \mathcal{F}^{-1}[e^{-j\omega t_0} e^{j\omega t_0}] = \mathcal{F}^{-1}[1] = \delta(t)$$

4.51 Find the equivalent frequency-domain filter transfer function for the Laplacian kernel shown in Fig. 3.45(a). Show that your result behaves as a highpass filter transfer function. (Hint: Study the solution to Problem 4.47.)

$$\begin{bmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{bmatrix}$$

$$\begin{aligned} \mathcal{H}(u, v) &= e^{j\pi \frac{u}{2}} + e^{j\pi \frac{v}{2}} + e^{-j\pi \frac{u}{2}} + e^{-j\pi \frac{v}{2}} - 4 \\ &= 2\cos\left(\frac{\pi u}{2}\right) + 2\cos\left(\frac{\pi v}{2}\right) - 4 \end{aligned}$$

at (0,0): $\mathcal{H}(0,0) = 0$. at (1.5, 1.5): $\mathcal{H}(1.5, 1.5) = -8 \rightarrow$ high-pass

4.47* Consider a 3×3 spatial kernel that averages the four closest neighbors of a point (x, y) , but excludes the point itself from the average.

- (a) Find the equivalent filter transfer function, $H(u, v)$, in the frequency domain.
 (b) Show that your result is a lowpass filter transfer function.

$$\begin{aligned} \mathcal{H}(u, v) &= \frac{1}{4} (e^{j\pi(0,u)} + e^{-j\pi(0,u)} + e^{-j\pi(u,0)} + e^{j\pi(u,0)}) \\ &= \frac{1}{4} (2\cos\left(\frac{\pi u}{2}\right) + 2\cos\left(\frac{\pi v}{2}\right)) \end{aligned}$$

$\therefore$ it depends on $\cos$, that acts as a low-pass filter

PART2:

Design a computer program for *spatial filtering* operations using various types of masks. Discuss the effect of mask size on the processed images and the computation time.

Smoothing filter (low-pass) :



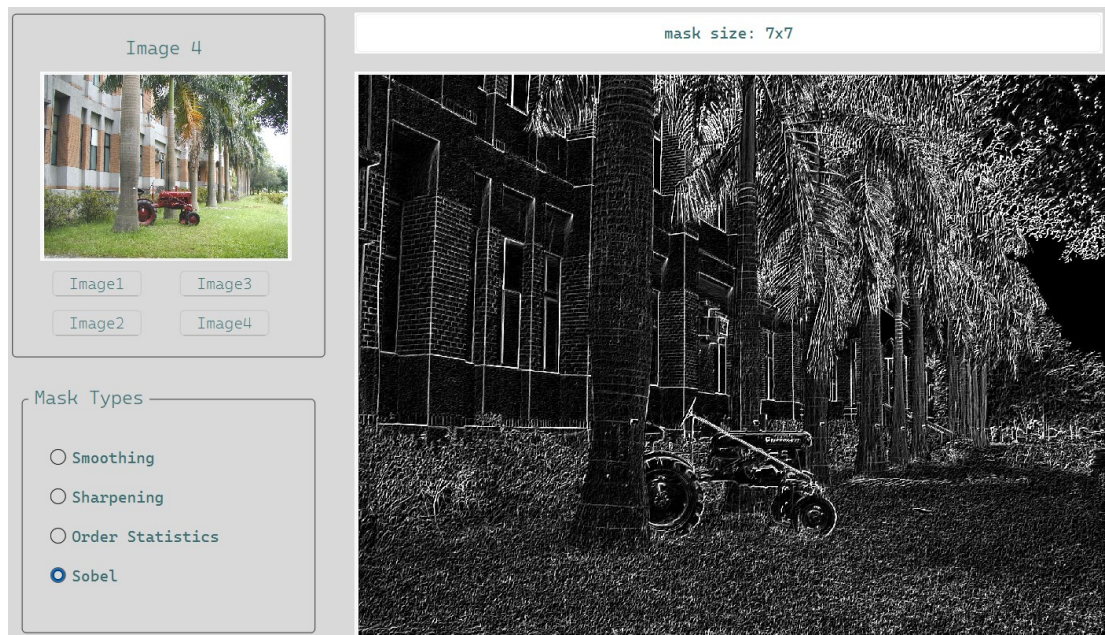
Sharpening filter (high-pass) :



Order Statistics filter (median) :



Sobel filter :



For smooth & sharpen filter :

```

QImage MainWindow::fil(const QImage& image, const QVector<QVector<double>>& kernel) {
    int padding = kernel.size() / 2;
    QImage padImage = pad(image, padding);
    QImage result = image;

    for (int y = 0; y < image.height(); ++y) {
        for (int x = 0; x < image.width(); ++x) {
            double sum = 0; //accumulate convolution result
            for (int ky = 0; ky < kernel.size(); ++ky) {
                for (int kx = 0; kx < kernel[0].size(); ++kx) {
                    int px = x + kx; //shift pixel by x-direction
                    int py = y + ky; //shift pixel in y-direction
                    sum += qGray(padImage.pixelColor(px, py).rgb()) * kernel[ky][kx]; //product of grayscaleValue and kernel
                }
            }
            int value = qBound(0, static_cast<int>(sum), 255); //clamp the result in [0, 255]
            result.setPixelColor(x, y, QColor(value, value, value));
        }
        ui->progressBar->setValue(y);
    }
    return result;
}

QImage MainWindow::pad(const QImage& image, int padding) {
    QImage padImage(image.width() + 2 * padding, image.height() + 2 * padding, image.format());
    padImage.fill(Qt::black);

    //copy the image to the center of padded image
    for (int y = 0; y < image.height(); ++y) {
        for (int x = 0; x < image.width(); ++x) {
            padImage.setPixelColor(x + padding, y + padding, image.pixelColor(x, y));
        }
    }

    //pad left & right edges
    for (int y = 0; y < padImage.height(); ++y) {
        for (int x = 0; x < padding; ++x) {
            padImage.setPixelColor(x, y, padImage.pixelColor(2 * padding - x, y));
            padImage.setPixelColor(padImage.width() - 1 - x, y, padImage.pixelColor(padImage.width() - 1 - 2 * padding + x, y));
        }
    }

    //pad up & down edges
    for (int x = 0; x < padImage.width(); ++x) {
        for (int y = 0; y < padding; ++y) {
            padImage.setPixelColor(x, y, padImage.pixelColor(x, 2 * padding - y));
            padImage.setPixelColor(x, padImage.height() - 1 - y, padImage.pixelColor(x, padImage.height() - 1 - 2 * padding + y));
        }
    }

    return padImage;
}

```

apply convolution with padded images, and adjust mask size for smoothing, mask size+coefficients for sharpening :

```

QImage MainWindow::smooth(const QImage& image, int size) {
    QVector<QVector<double>> kernel(size, QVector<double>(size, 1.0 / (size * size)));
    return fil(image, kernel);
}

QImage MainWindow::sharp(const QImage& image, int size, double coefficient) {
    QVector<QVector<double>> kernel(size, QVector<double>(size, -coefficient / (size * size)));
    kernel[size / 2][size / 2] += 1 + coefficient; //increase center weight
    return fil(image, kernel);
}

```

For median filter :

```

QImage MainWindow::order(const QImage& image, int size) {
    int padding = size / 2;
    QImage padImage = pad(image, padding);
    QImage result = image;

    for (int y = 0; y < image.height(); ++y) {
        for (int x = 0; x < image.width(); ++x) {
            std::vector<int> values;
            for (int dy = -padding; dy <= padding; ++dy) {
                for (int dx = -padding; dx <= padding; ++dx) {
                    values.push_back(qGray(padImage.pixelColor(x + padding + dx, y + padding + dy).rgb())); //get grayscaleValue of neighbor
                }
            }
            std::nth_element(values.begin(), values.begin() + values.size() / 2, values.end()); //median values
            int median = values[values.size() / 2];
            result.setPixelColor(x, y, QColor(median, median, median));
        }
        ui->progressBar->setValue(y);
    }
    return result;
}

```

For sobel filter :

```
QImage MainWindow::sobel(const QImage& image) {
    QVector<QVector<double>> kernelX = {{-1, 0, 1}, {-2, 0, 2}, {-1, 0, 1}}; //x-kernel
    QVector<QVector<double>> kernelY = {{-1, -2, -1}, {0, 0, 0}, {1, 2, 1}}; //y-kernel

    QImage gradX = fil(image, kernelX);
    QImage gradY = fil(image, kernelY);

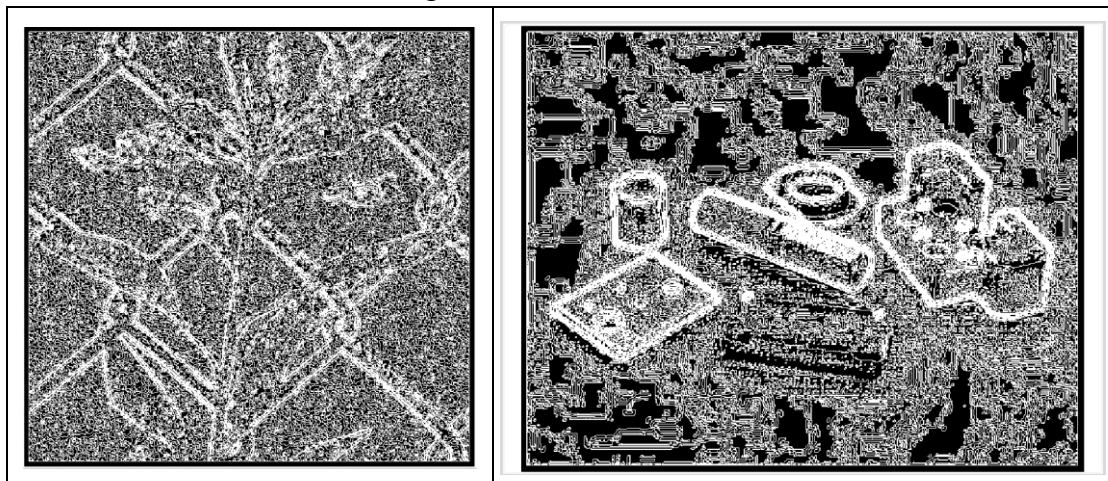
    QImage result = image;
    for (int y = 0; y < image.height(); ++y) {
        for (int x = 0; x < image.width(); ++x) {
            int gx = qGray(gradX.pixelColor(x, y).rgb()); //x-gradient
            int gy = qGray(gradY.pixelColor(x, y).rgb()); //y-gradient
            int magnitude = qMin(255, static_cast<int>(std::sqrt(gx*gx + gy*gy))); //gradient magnitude
            result.setPixelColor(x, y, QColor(magnitude, magnitude, magnitude)); //set gradient magnitude to pixel
        }
        ui->progressBar->setValue(y);
    }
    return result;
}
```

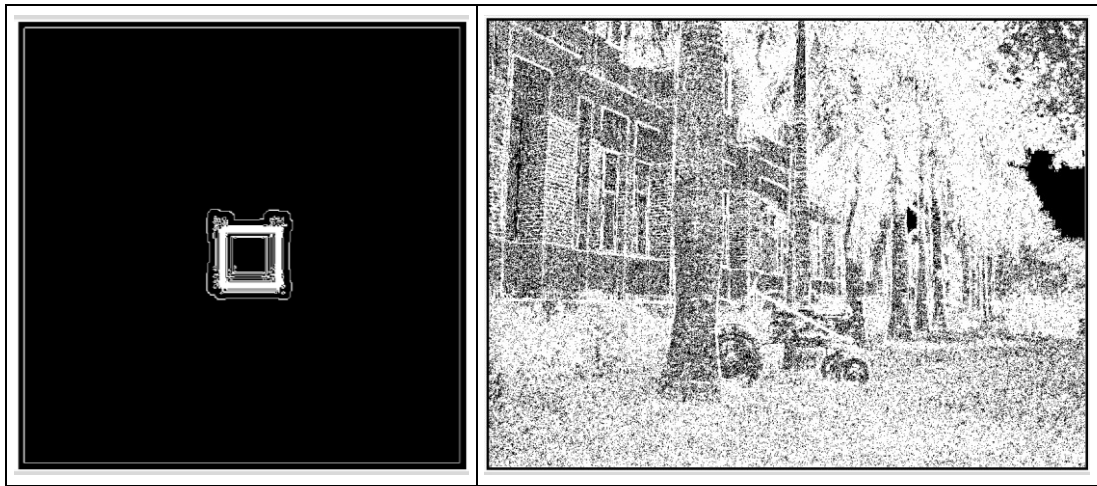
PART3:

The Marr-Hildreth edge detection method operates by convolving the image with the Laplacian of Gaussian operators, which is a second derivative of a Gaussian filter.

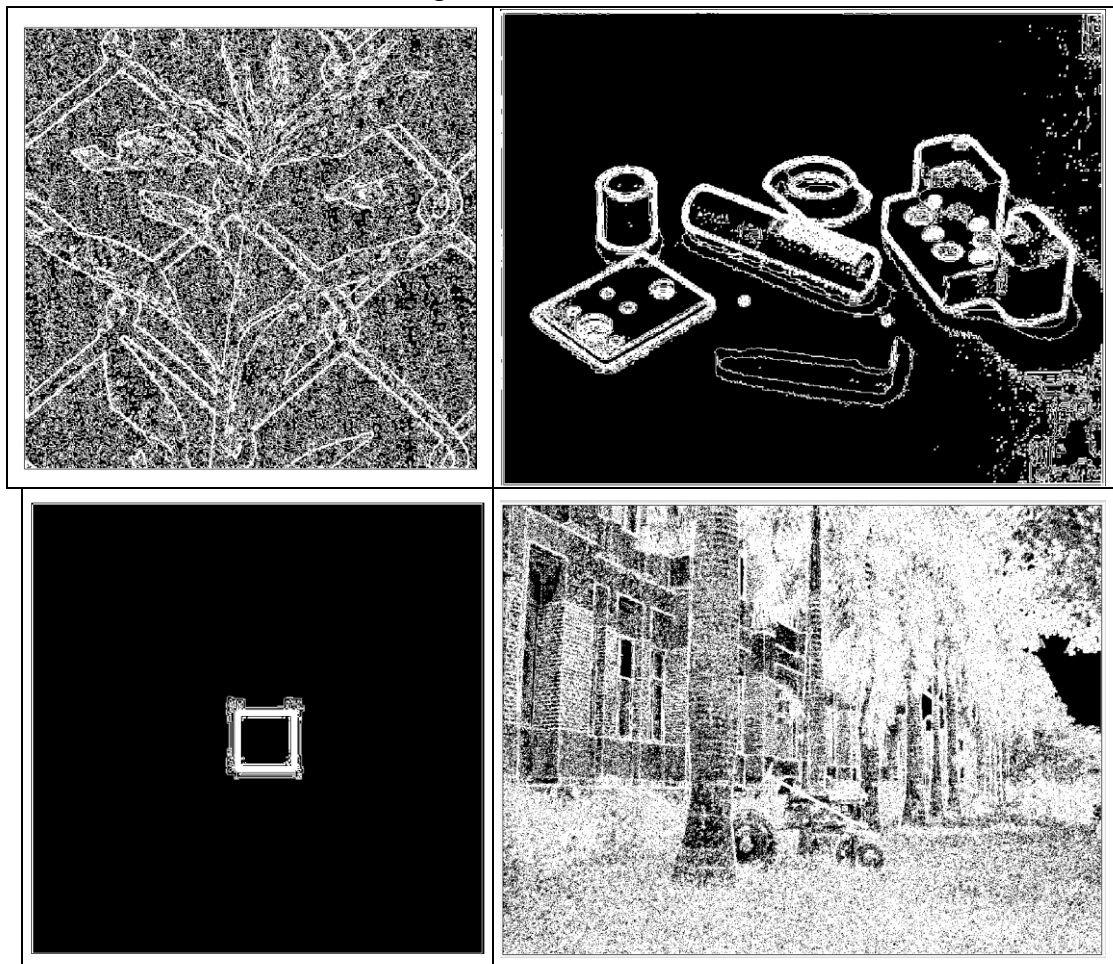
1. Smooth the image with a Gaussian filter
2. Convolve the image with a Laplacian mask.
3. Discuss the effect of zero-crossing threshold on the Marr-Hildreth edge detection method.

sigma=1, kernel size=15

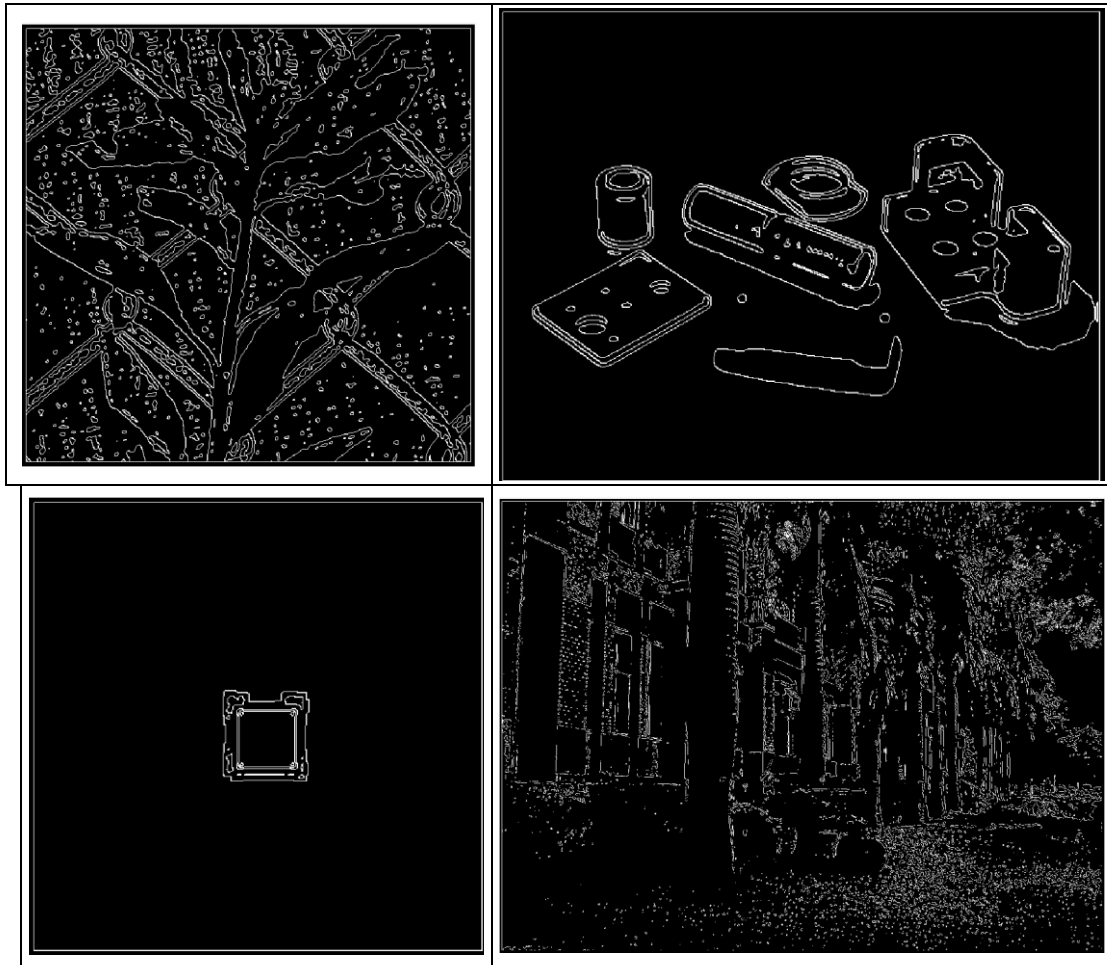




sigma=1, kernel size=5



sigma=3, kernel size=13



PART4:

Implement a computer program following the *local enhancement method*.

1. Reproduce the enhanced image (image 4-1.jpg)
2. Process the original images (image 4-1.jpg, image 4-2.jpg) with the local enhancement method and compare the results using histogram equalization method
3. Discuss the effect of neighborhood region size, S_{xy}

The key parameters:

mG: global mean (average intensity of entire image)

k_0, k_1 : constants (threshold multipliers)

sG: global standard deviation (intensity variation)

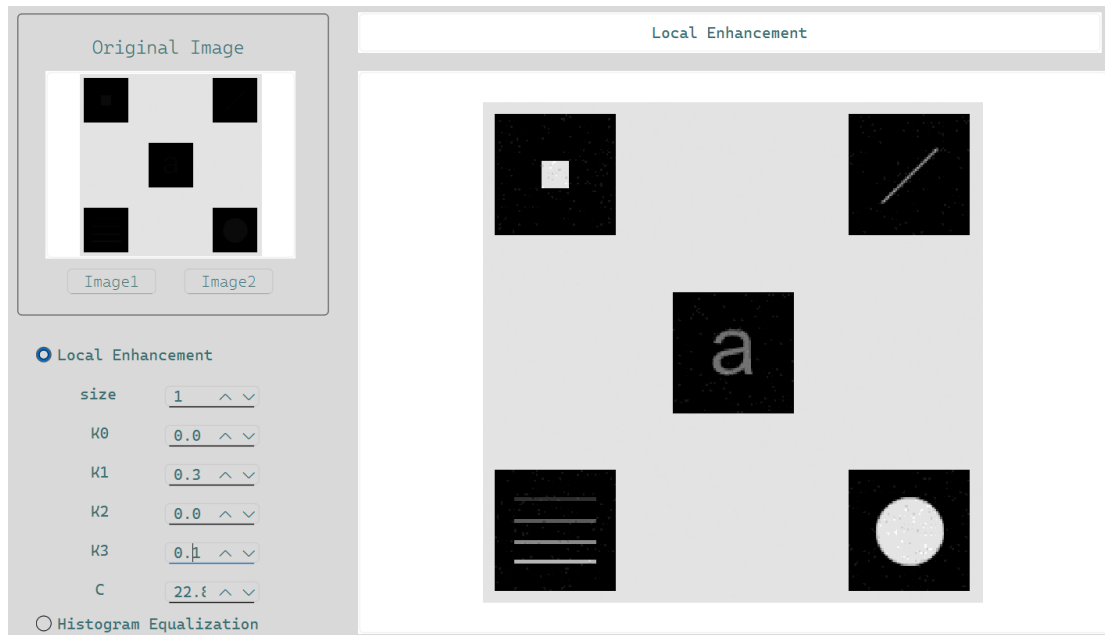
k_2, k_3 : constants (threshold multipliers)

C: intensity (multiplier for enhancement)

The process procedure:

- a. Padding the image
- b. Calculating global and local mean and standard deviation
- c. Apply for each pixel
- d. Enhance if the local statistics match the thresholds

Local Enhancement:



Implement the function to apply local enhancement method to the image based on local statistics (mean and standard deviation) compared to global statistics.

Results depend on the chosen parameters (lSize, k0, k1, k2, k3, c). Enhance details in both bright and dark regions.

$$\begin{cases} k_0 \cdot mG \leq mS_{xy} \leq k_1 \cdot mG \\ k_2 \cdot sG \leq sS_{xy} \leq k_3 \cdot sG \end{cases}$$

If both conditions meet, enhance the pixel intensity:

$$g(x, y) = C \cdot f(x, y)$$

If the conditions doesn't meet, leave the pixel unchanged:

$$g(x, y) = f(x, y)$$

```

QImage MainWindow::localEn(const QImage& image, int size, double k0, double k1, double k2, double k3, double c){
    int padding = size / 2;
    QImage padImage = pad(image, padding);
    QImage result = image.copy();

    double mG = 0.0;          //global mean
    double sG = 0.0;          //global standard deviation
    int totalPixels = image.width() * image.height();

    for (int y = 0; y < image.height(); ++y) {
        for (int x = 0; x < image.width(); ++x) {
            int pixel = qGray(image.pixel(x, y));
            mG += pixel;
            sG += pixel * pixel;
        }
    }
    mG /= totalPixels;
    sG = sqrt(sG / totalPixels - mG * mG);

    for (int y = 0; y < image.height(); ++y) {
        for (int x = 0; x < image.width(); ++x) {
            double mL = 0.0;          //local mean
            double vL = 0.0;          //local variance

            for (int dy = -padding; dy <= padding; ++dy) {
                for (int dx = -padding; dx <= padding; ++dx) {
                    int pixel = qGray(padImage.pixel(x + padding + dx, y + padding + dy));
                    mL += pixel;
                    vL += pixel * pixel;
                }
            }
            mL /= (size * size);
            vL = (vL / (size * size)) - (mL * mL);
            double sL = sqrt(vL);          //local standard deviation

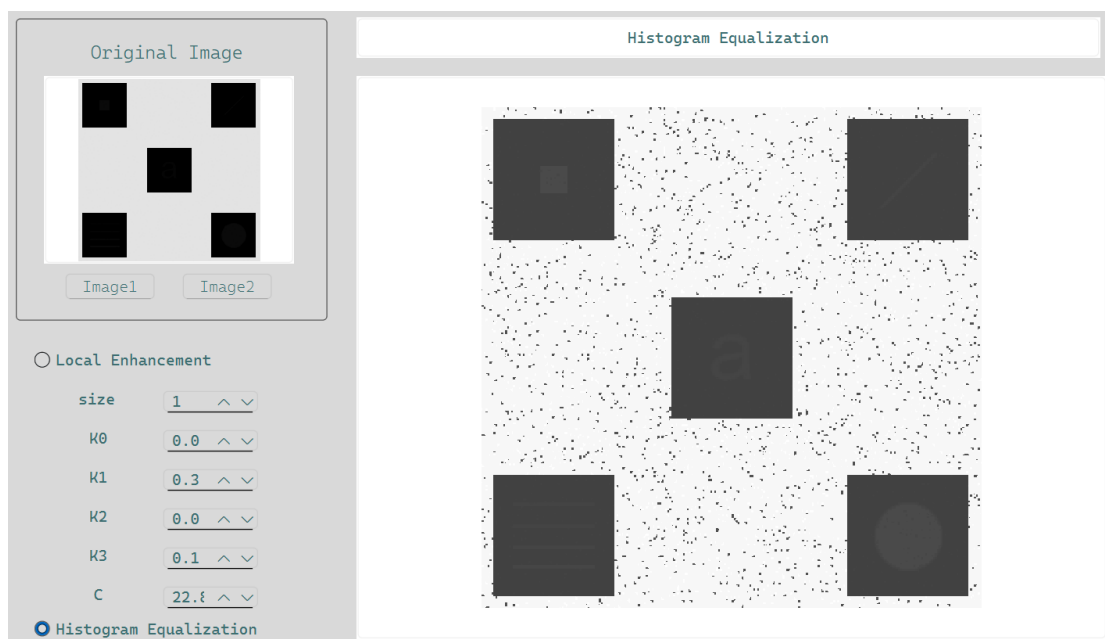
            int pixel = qGray(image.pixel(x, y));
            double enhance = pixel;

            if (k0 * mG <= mL && mL <= k1 * mG &&
                k2 * sG <= sL && sL <= k3 * sG) {
                enhance = pixel * c;
            }

            int value = qBound(0, qRound(enhance), 255);
            result.setPixelColor(x, y, QColor(value, value, value));
        }
        ui->progressBar2->setValue(y);
    }
    return result;
}

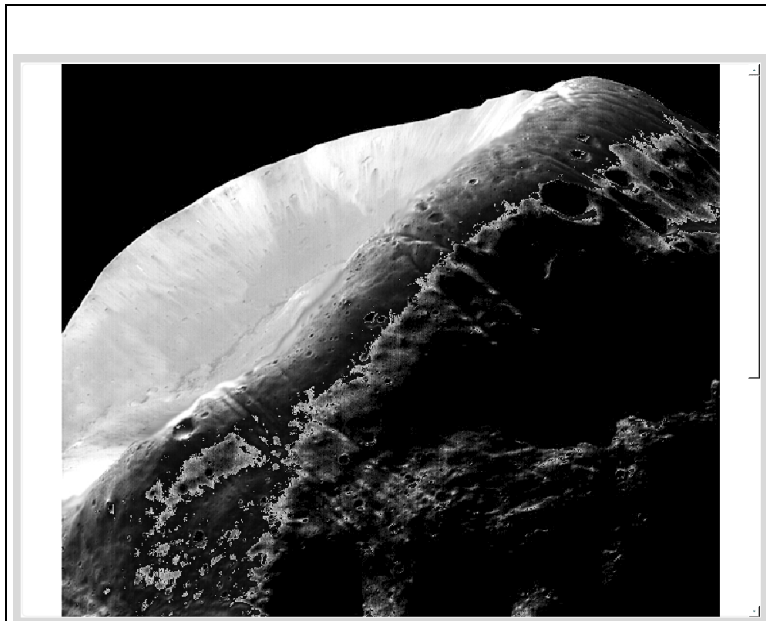
```

Histogram Equalization:

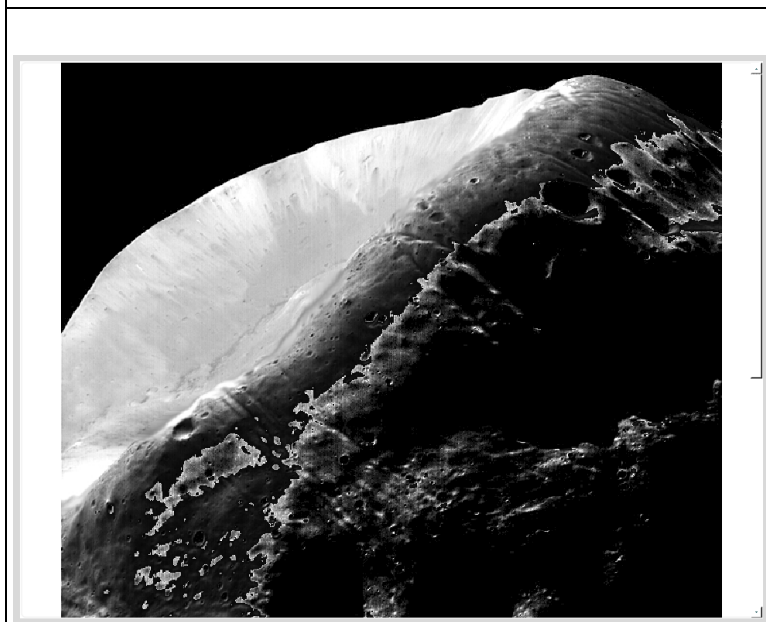


Enhances global contrast by the most frequent intensity values, may over-enhance the image, leading to loss of details in very bright or dark areas.

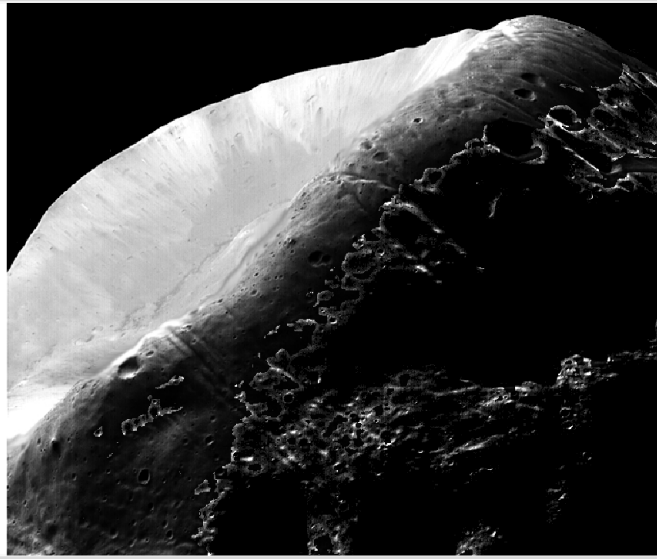
Effect of neighborhood region size (S_{xy}):



Mask size = 1



Mask size = 33



Mask size = 81